

CUSTOM PAYLOAD ENCODER & OBFUSCATION FRAMEWORK

PROJECT PRESENTATION

Custom Payload Encoder & Obfuscation Framework
Domain: Cyber Security and Ethical Hacking
Presented By: UMMEDA VIVEK
Project Type: Cyber Security Project
Tools Used: Python and Linux



INTRODUCTION

The Custom Payload Encoder & Obfuscation Framework is a cybersecurity project designed to demonstrate encoding, decoding, string obfuscation, and simulated pattern detection.The project helps users understand how different encoding techniques transform text and how predefined detection rules identify specific patterns.The framework is developed for educational and authorized cybersecurity testing purposes.



PROBLEM STATEMENT

Text representation and pattern matching are important concepts in cyber security. Different encoding techniques change the representation of a string, while simple pattern detectors search for predefined pattern. This project demonstrates these concepts through controlled test cases and compares the detection results before and after string transformations.



PROJECT OBJECTIVES

- To implement Base64 encoding and decoding.
- To demonstrate XOR encoding and decoding.
- To implement ROT13 transformation.*
- To demonstrate string obfuscation techniques.*
- To simulate predefined pattern detection.
- To compare original and transformed strings.
- To record testing results and screenshots.

TOOLS AND TECHNOLOGIES USED

- Programming Language: Python
- Operating System: Linux / Kali Linux
- Encoding Techniques:*

Base64

XOR

ROT13

- Other Techniques:

String Obfuscation

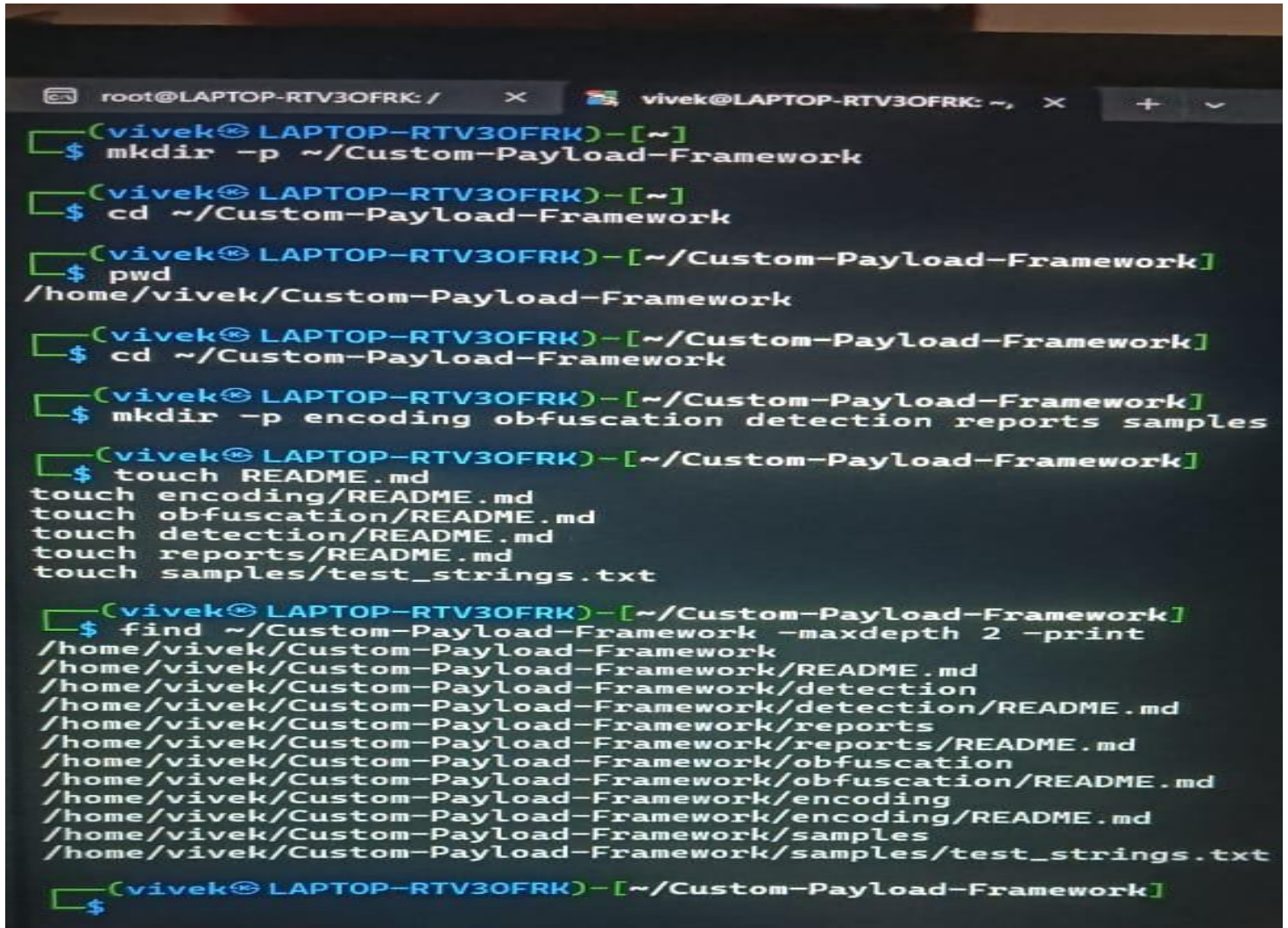
Pattern Matching

Detection Simulation

- Documentation:

Microsoft PowerPoint

Project Screenshots





The project follows a modular workflow

1. User enters a test method is selected.
3. The input is encoded or transformed.
4. The pattern detection simulation is executed.
5. The results are displayed.
6. The results are recorded for documentation.

ARCHITECTURE DIAGRAM

User Input

⬇

Method Selection

⬇

Encoding / Transformation

⬇

Pattern Detection Simulation

⬇

Result Comparison

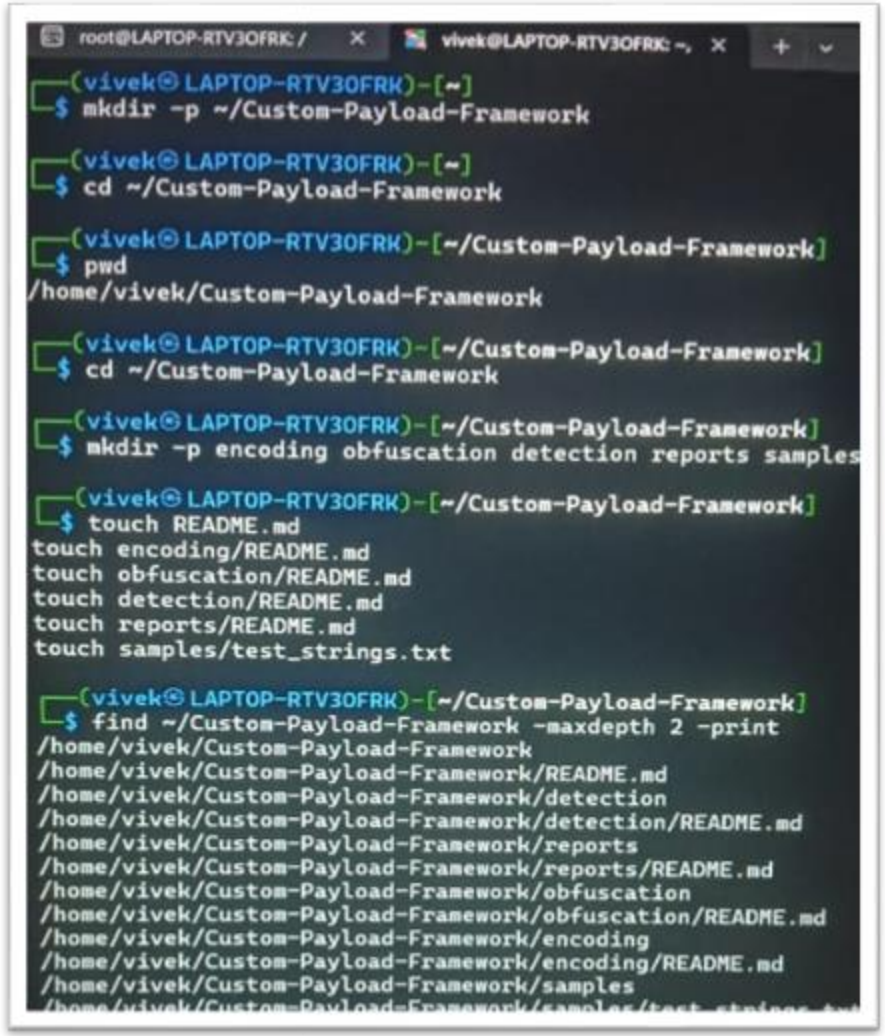
⬇

Final Test Results

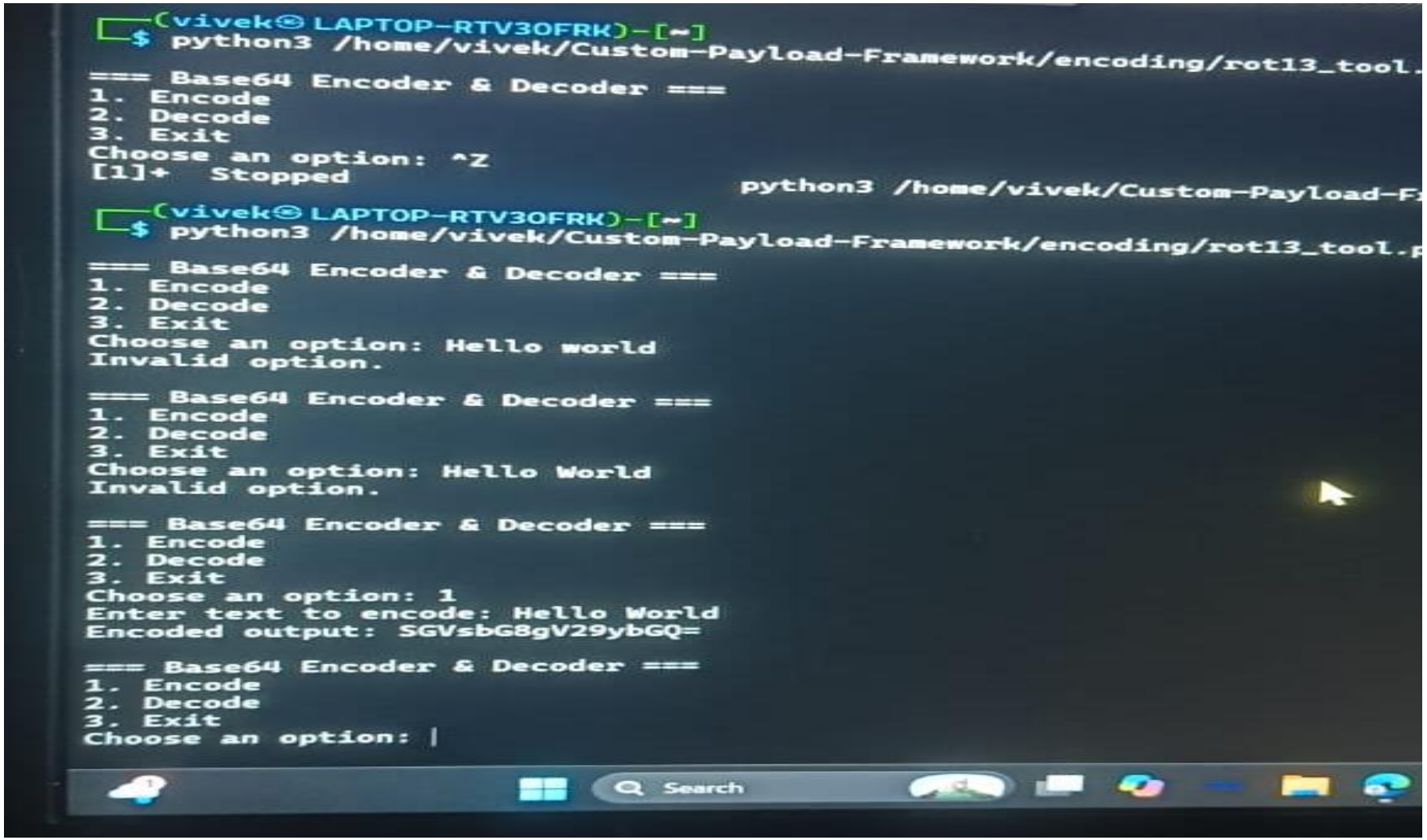
UNIFIED MENTOR

IMPLEMENTATION

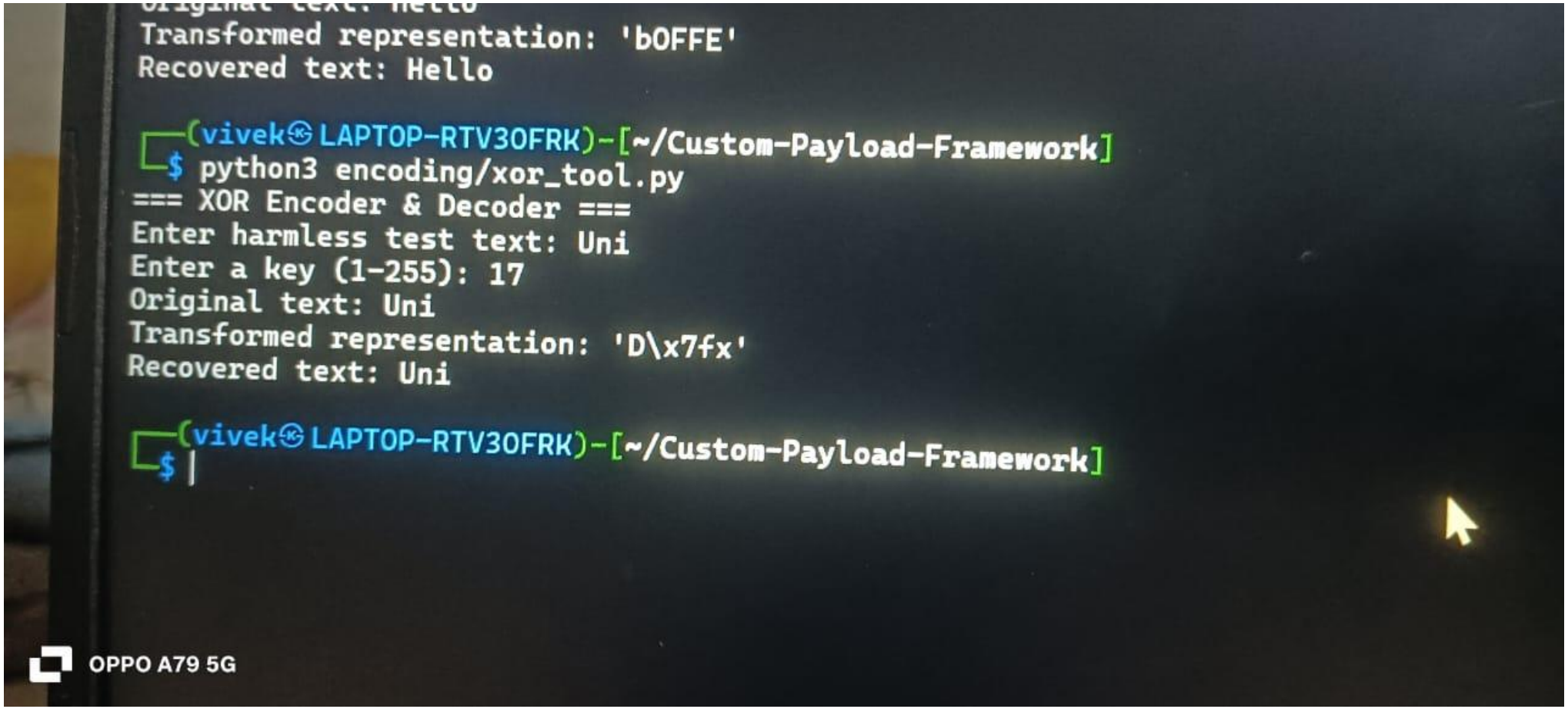
- The framework consists of encoding, decoding, string transformation, and detection simulation components.
- Base64, XOR, and ROT13 are used to demonstrate different string transformations .
- The obfuscation module demonstrates changes in string representation .
- The detection simulation checks predefined patterns and displays the corresponding results.



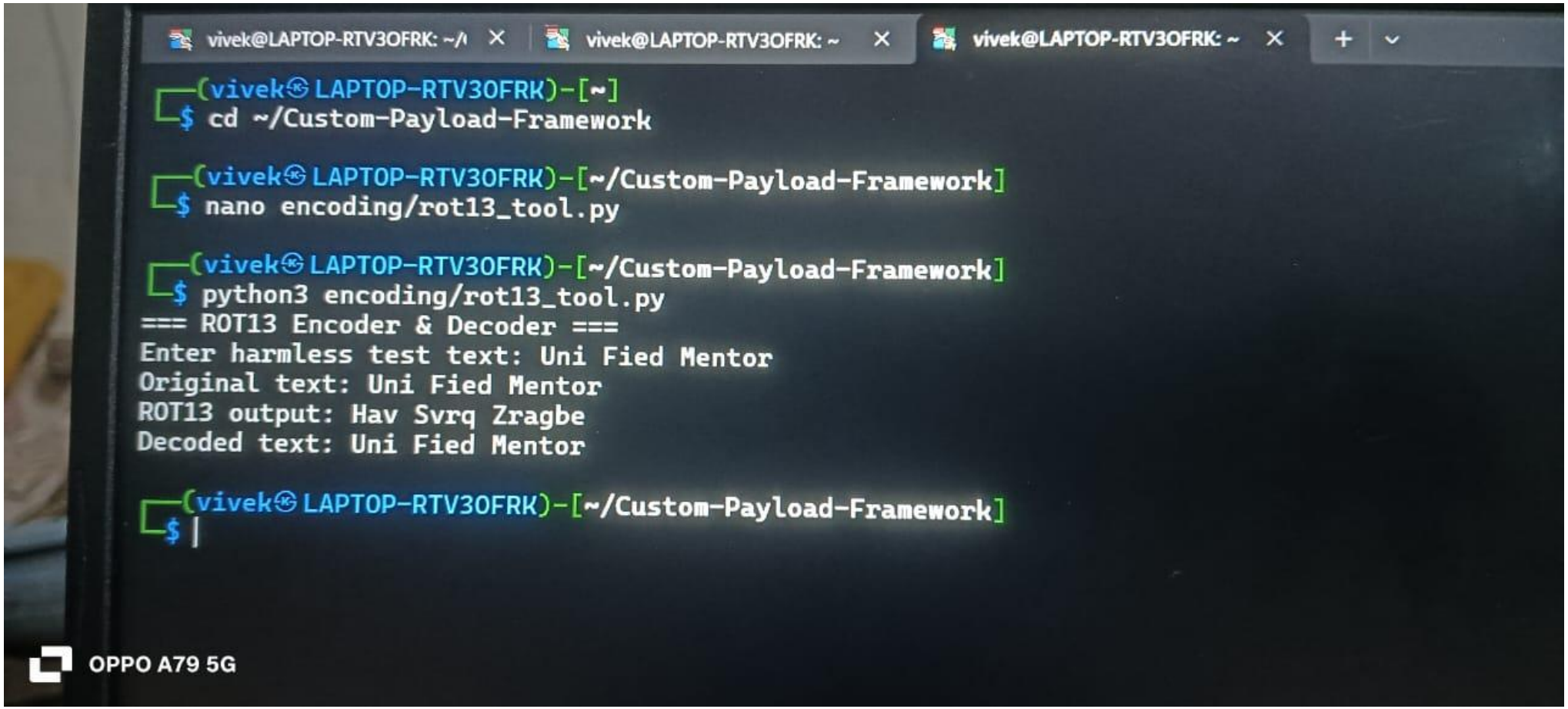
BASE 64 INPUT AND OUT PUT



XOR ENCODER & DECODER



ROT13 INPUT AND OUTPUT

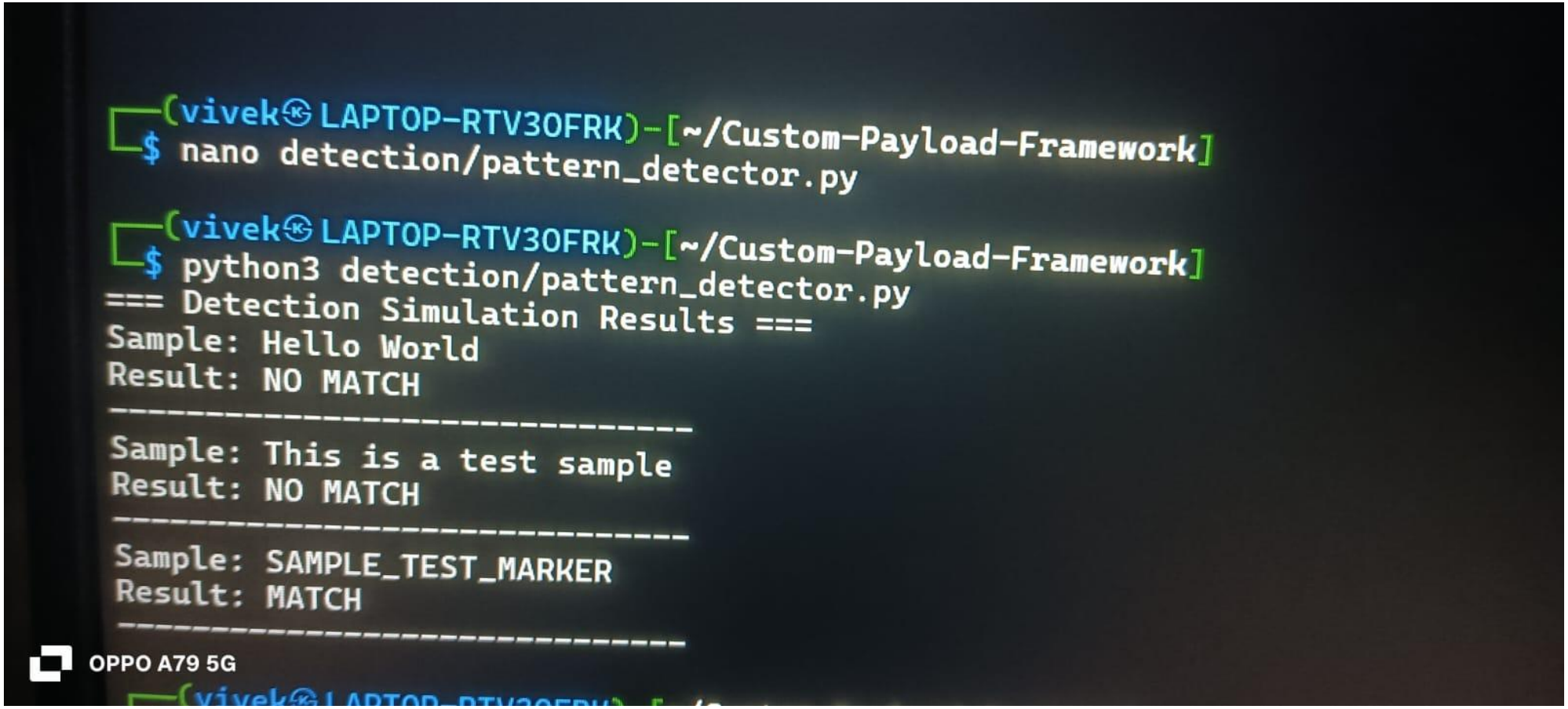




TESTING AND RESULTS

- The framework was tested using sample strings .
- The detection simulation checks whether a predefined pattern exists in the input .
- Test results are displayed as MATCH or NO MATCH .
- The results demonstrate the behavior of the implemented pattern-matching logic.

RESULTS


┌──(vivek㉿LAPTOP-RTV3OFRK)-[~/Custom-Payload-Framework]
└─$ nano detection/pattern_detector.py
┌──(vivek㉿LAPTOP-RTV3OFRK)-[~/Custom-Payload-Framework]
└─$ python3 detection/pattern_detector.py
=== Detection Simulation Results ===
Sample: Hello World
Result: NO MATCH

Sample: This is a test sample
Result: NO MATCH

Sample: SAMPLE_TEST_MARKER
Result: MATCH

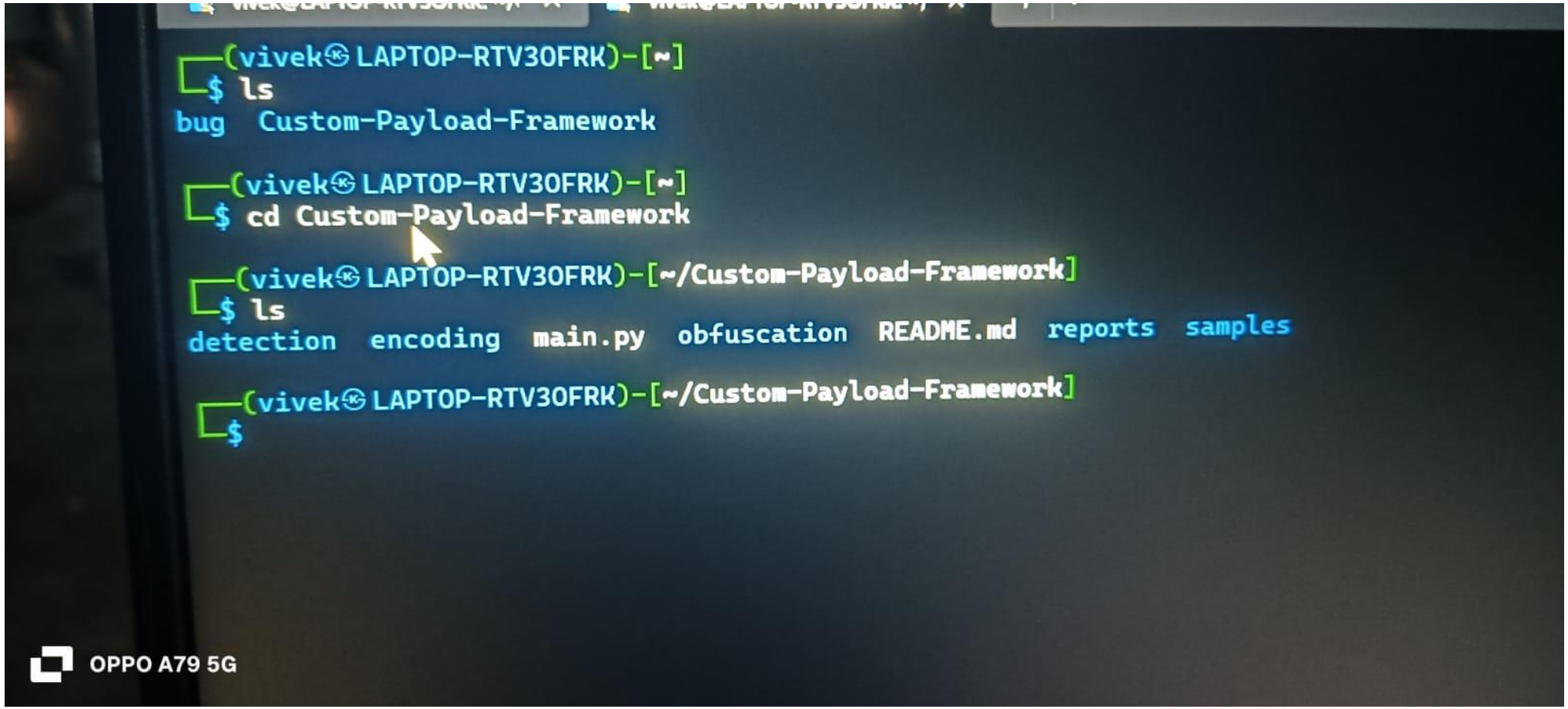
OPPO A79 5G



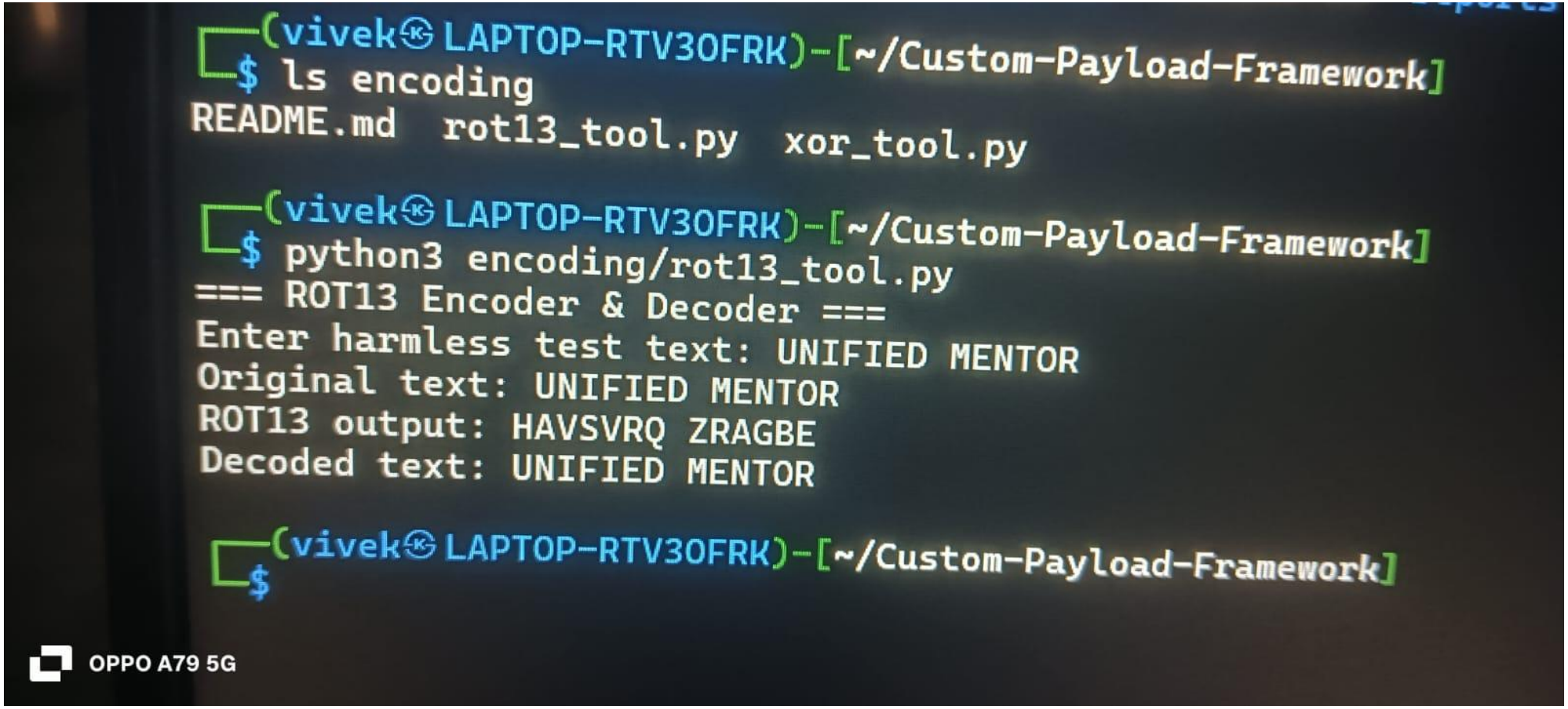
TESTING EVIDENCE

- The following screenshots provide evidence of project implementation and testing.
- 1. Project Directory Structure
- 2. Encoding and Decoding Output
- 3. String Obfuscation Output
- 4. Pattern Detection Results
- 5. Successful Execution of the Framework

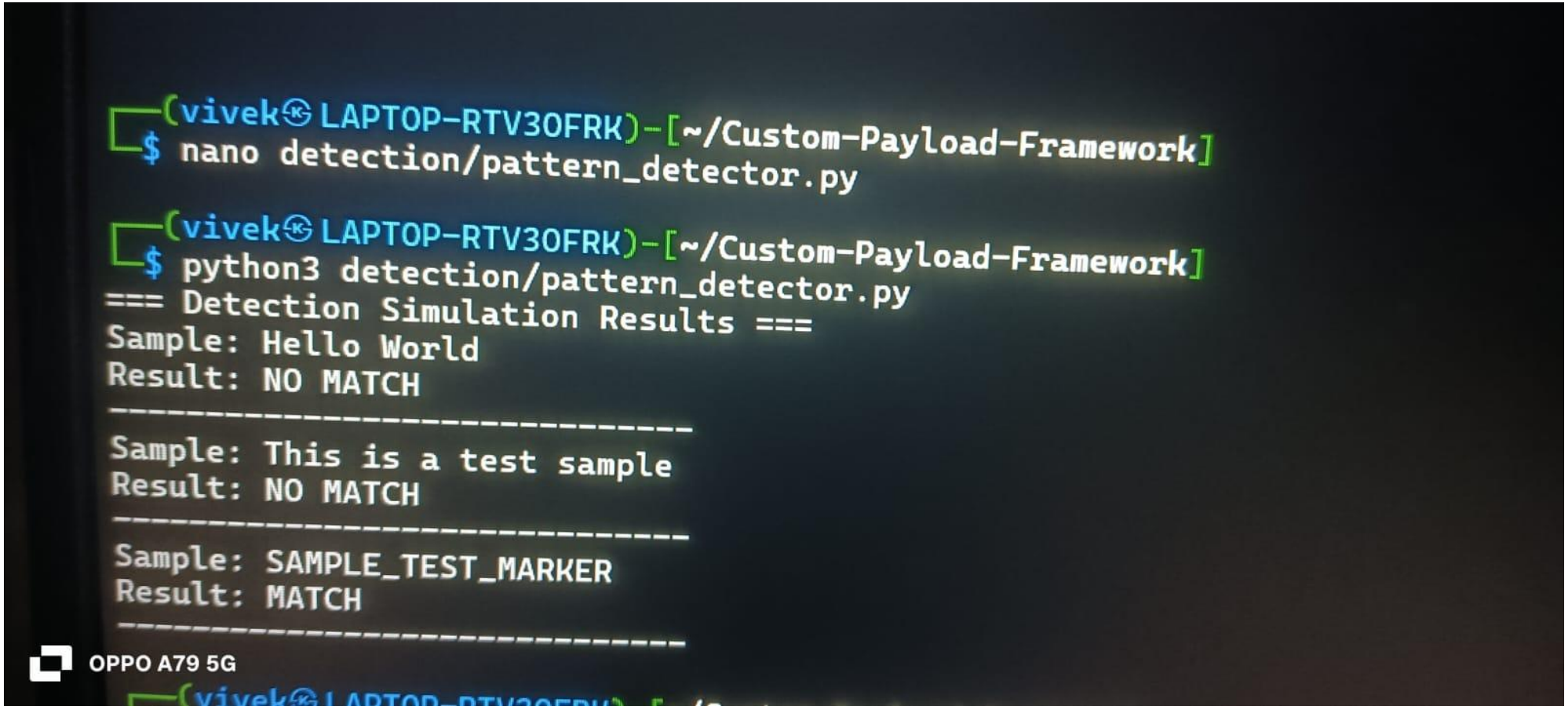
PROJECT FOLDER



ENCODING OUTPUT



DECTECTION OUTPUT





CONCLUSION AND FUTURE SCOPE

- The Custom Payload Encoder & Obfuscation Framework demonstrates encoding, decoding, string obfuscation, and simulated pattern detection .
- The project provides practical knowledge of string transformations and basic pattern-matching techniques .
- Future Scope:
- Add more encoding and decoding methods.
- Improve the testing and reporting modules.
- Develop a user-friendly interface.
- Extend testing with additional harmless sample strings.



THANK YOU